Mi Primera Presentación

Un template básico de Beamer

Tu Nombre

Tu Institución

19 de marzo de 2026

Contenido

1. Generación del Dataset Simulado
2. Normalización de las labels
3. Pipeline de Entrenamiento

El problema: Medir la geometría de los poros

Material: AAO

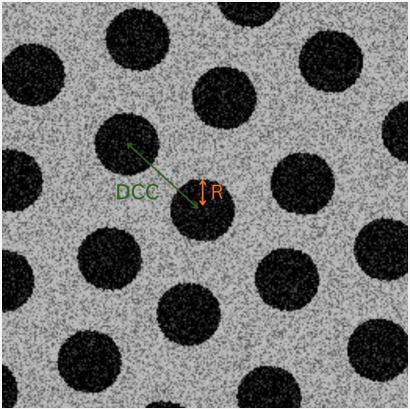
Membrana nanoestructurada con poros cilíndricos organizados en red hexagonal. Sus propiedades ópticas y mecánicas dependen de la geometría de los poros.

Objetivo

Predecir automáticamente los parámetros geométricos de una imagen SEM.

5 variables objetivo

R, σ_R	Radio promedio y dispersión [px]
DCC, σ_{DCC}	Distancia centro a centro [px]
P	Porosidad (%)

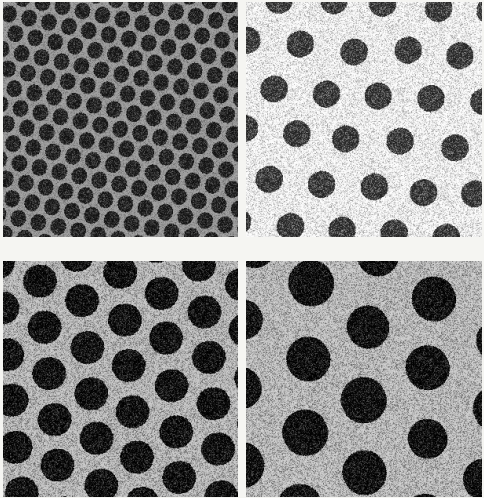


Generación del Dataset Simulado

Parámetros aleatorios por imagen

Parámetro	Rango
Radio r	$[10,\ 30]$ px
Distancia dcc	$[2{,}5r,\ 4{,}5r]$
Desorden en r, dcc	hasta 10 %
Rotación de red	$[0^\circ,\ 360^\circ]$
Brillo B	$[-40,\ 40]$
Contraste C	$[0{,}6,\ 1{,}4]$
Ruido	nivel 1

Se generaron **20 000 imágenes** de 300×300 px. Cada imagen genera una fila en el CSV con variables: R, σ_R, DCC, σ_{DCC}, P.



Normalización de las Labels

Por qué normalizar

Las variables tienen escalas muy distintas: $R \sim 20$ px pero $P \sim 0{,}3$. Sin normalizar, las variables grandes dominan el MAE y el modelo aprende mal.

StandardScaler

Técnica que transforma las variables:

$$x_{norm} = \frac{x - \mu}{\sigma}$$

donde μ es media y σ es desviación estándar.
Resultado: media 0 y desv. est. 1.

Transformación Inversa

Despejar x de la ecuación:

$$x = x_{norm} \cdot \sigma + \mu$$

Importante: Los valores de μ y σ se guardan.

Flujo del proceso

1. CSV Original con 5 variables
2. Aplicar StandardScaler
3. Guardar scaler
4. Labels listas para entrenar
5. Entrenar la red neuronal
6. Usar Scaler Inverso para obtener MAPE

Transformación de Imágenes

Valores originales

Las imágenes SEM vienen con píxeles en rango $[0, 255]$.

Normalización de píxeles

Se transforman a uno de estos rangos:

- Rango $[0, 1]$: dividir por 255
- Rango $[-1, 1]$: normalizar a media 0 y desv. est. 1
- Mantener $[0, 255]$: sin transformación

Por qué

Facilita convergencia del modelo y evita dominancia numérica de valores grandes.

División del Dataset

Partición

Las 20 000 imágenes se dividen en:

- **Train Set**: entrena el modelo
- **Validation Set**: valida en cada época
- **Test Set**: evalúa al final

Batching

Las imágenes se agrupan en lotes para:

- Optimizar memoria GPU/CPU
- Acelerar entrenamiento
- Mejorar estabilidad numérica

Entrenamiento del Modelo

Arquitectura

Dos opciones:

- **CNN Completa**: red convolucional entrenada desde cero
- **Transfer Learning**: últimas capas convolucionales de modelo preentrenado

Datos de entrada

- Imágenes: transformadas a $[0, 1]$, $[-1, 1]$ o $[0, 255]$
- Labels: normalizados con StandardScaler (guardado en `scaler.pkl`)

Proceso

Minimizar pérdida en Train Set validando en cada época.

Obtención del MAPE en Test Set

Paso 1: Predicciones

El modelo entrenado realiza predicciones en Test Set:

$$\text{Modelo}(x_{imagen}) = (\hat{R}_{norm}, \hat{\sigma_{R\,norm}}, \hat{DCC}_{norm}, \hat{\sigma_{DCC\,norm}}, \hat{P}_{norm})$$

Paso 2: Scaler Inverso

Recuperar labels al espacio original usando `scaler.pkl`:

$$(\hat{R}, \hat{\sigma_R}, \hat{DCC}, \hat{\sigma_{DCC}}, \hat{P}) = \text{scaler.inverse_transform}(\hat{R}_{norm}, \hat{\sigma_{R\,norm}}, \ldots)$$

Paso 3: Calcular MAPE

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\,\%$$

donde y_i son los labels reales del Test Set.

Evaluación en Dataset Externo Real

Datos externos

Dataset de imágenes SEM **reales** (no simuladas) para evaluar generalización.

Función: evaluate_external_dataset

1. Cargar imágenes SEM reales
2. Aplicar misma transformación de píxeles
3. Realizar predicciones con modelo entrenado
4. Aplicar Scaler Inverso (`scaler.pkl`)
5. Calcular MAPE

Resultado

MAPE en datos reales indica rendimiento en datos no vistos.

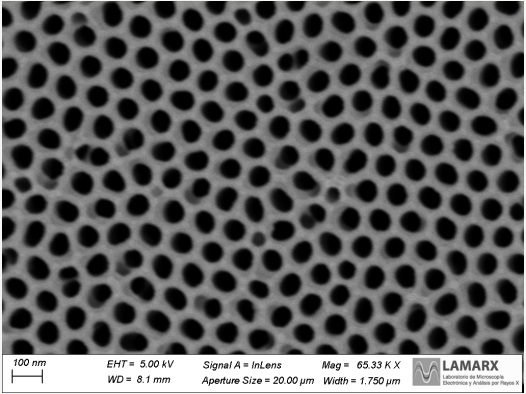


Imagen SEM real

¡Gracias!

Preguntas e inquietudes